

AI Researcher Technical Assessment Report

Cover Page

AI Researcher Technical Assessment

Comparative Evaluation of Large Language Models and Development of a Multi-Agent AI System using LangGraph

Prepared By

Sahil Bhanushali

Data Science Engineering Student

Date: 15 July 2026

Table of Contents

1. Introduction
 2. Project Objectives
 3. Phase 1 – LLM Research & Evaluation
 4. Benchmark Methodology
 5. Comparative Analysis
 6. Phase 2 – Multi-Agent AI Proof of Concept
 7. System Architecture
 8. Implementation
 9. Results and Findings
 10. Conclusion
 11. Future Scope
 12. References
-

1. Introduction

Large Language Models (LLMs) have become one of the most influential technologies in Artificial Intelligence, enabling machines to understand, generate, and reason over human language. Organizations increasingly use LLMs for customer support, software development, business automation, and decision support systems.

There are two major categories of LLMs:

- Closed-source models developed and hosted by commercial organizations.
- Open-source models that provide greater flexibility and customization.

This project evaluates both categories by comparing Google Gemini (closed-source) with Llama 3.1 (open-source via Groq). In addition, a Multi-Agent AI system was developed using LangGraph to demonstrate collaborative AI problem-solving in a pharmaceutical logistics use case.

The project aims to analyze model capabilities while showcasing how multiple AI agents can coordinate to automate business decision-making.

2. Project Objectives

General Objective

To evaluate different Large Language Models and develop a Multi-Agent AI workflow capable of solving logistics disruption scenarios through collaborative AI agents.

Specific Objectives

- Compare Gemini and Llama 3.1 across multiple evaluation tasks.
 - Assess reasoning, context retention, and structured output generation.
 - Design a Multi-Agent architecture using LangGraph.
 - Develop specialized AI agents for logistics decision-making.
 - Demonstrate workflow orchestration using a shared state.
 - Generate automated execution reports for shipment management.
-

3. Phase 1 – LLM Research & Evaluation

The first phase focused on evaluating the performance of two Large Language Models:

**Closed
Source**

Open Source

Google Gemini Llama 3.1 8B Instant (Groq)

Both models were tested using identical prompts to ensure a fair comparison.

Evaluation categories included:

- General Question Answering
 - Multi-turn Conversations
 - Structured JSON Generation
 - Reasoning Tasks
 - Context Retention
 - Response Consistency
-

4. Benchmark Methodology

A series of benchmark prompts were designed to evaluate the capabilities of each model.

Test 1 – General Question Answering

Both models were asked factual and explanatory questions to evaluate response quality and accuracy.

Test 2 – Multi-turn Conversation

A conversation consisting of multiple sequential prompts was used to measure context retention and conversational memory.

Test 3 – Structured JSON Output

Models were instructed to generate structured JSON responses representing logistics information.

Test 4 – Reasoning Task

The models were evaluated on their ability to analyze multiple delivery routes while considering factors such as transportation cost, estimated arrival time, and customer priority.

5. Comparative Analysis

Based on the conducted experiments, the following observations were made:

Evaluation Criteria	Google Gemini	Llama 3.1 (Groq)
General Knowledge	Excellent	Excellent
Reasoning	Excellent	Very Good
Multi-turn Context	Excellent	Very Good
JSON Formatting	Excellent	Excellent
Response Speed	Good	Excellent
Cost Efficiency	Paid Tier	Low Cost / Fast API

Overall, both models performed well across the benchmark tasks. Gemini demonstrated slightly stronger contextual reasoning, while Llama 3.1 provided significantly faster responses and excellent structured output generation, making it highly suitable for real-time AI applications.

6. Phase 2 – Multi-Agent AI Proof of Concept

A Multi-Agent AI workflow was developed to automate logistics disruption management for pharmaceutical shipments.

The system was implemented using LangGraph, allowing multiple AI agents to collaborate through a shared workflow state.

Business Problem

Pharmaceutical shipments often experience delays due to traffic, weather conditions, or operational issues. Delays can affect temperature-sensitive products such as vaccines and medicines.

The developed system assists logistics managers by automatically analyzing disruptions, generating alternative delivery routes, selecting the optimal route, and producing execution reports.

7. System Architecture

The Multi-Agent workflow consists of four specialized AI agents.

Agent 1 – Alert Analysis Agent

Responsibilities:

- Analyze shipment delay information
 - Identify delay reason
 - Determine severity level
 - Recommend immediate actions
-

Agent 2 – Route Planning Agent

Responsibilities:

- Generate alternative routes
 - Estimate transportation cost
 - Calculate ETA
 - Recommend suitable routes
-

Agent 3 – Decision Agent

Responsibilities:

- Compare available routes
- Evaluate cost, ETA, and operational risk
- Select the most efficient delivery route

Agent 4 – Execution Agent

Responsibilities:

- Generate execution report
 - Notify driver
 - Notify warehouse
 - Notify customer
 - Update shipment status
-

8. Implementation

The system was developed using the following technologies:

Component	Technology
Programming Language	Python
Development Platform	Google Colab
Workflow Engine	LangGraph
AI Framework	LangChain
Large Language Model	Llama 3.1 8B Instant
API Provider	Groq

A shared state object was created using LangGraph to enable communication between agents.

Each AI agent receives the current workflow state, performs its assigned task, updates the shared information, and passes control to the next agent.

This modular architecture improves scalability and allows new AI agents to be integrated with minimal changes.

9. Results and Findings

The developed Multi-Agent system successfully completed the logistics workflow.

Observed outputs included:

- Successful shipment delay analysis.
- Identification of disruption severity.
- Generation of multiple alternative delivery routes.
- Selection of the optimal route.
- Automatic execution report generation.

The workflow demonstrated smooth coordination between all four AI agents using LangGraph's state management capabilities.

Key findings include:

- Multi-Agent systems effectively decompose complex business problems.
 - LangGraph simplifies workflow orchestration.
 - Llama 3.1 generated accurate and context-aware responses.
 - Shared state architecture enabled seamless communication among AI agents.
-

10. Conclusion

This project successfully demonstrated both comparative LLM evaluation and the implementation of a Multi-Agent AI workflow.

The benchmarking phase highlighted the strengths of Google Gemini and Llama 3.1 across multiple evaluation criteria. While Gemini showed excellent contextual reasoning, Llama 3.1 offered fast inference and reliable structured outputs suitable for enterprise automation.

The Multi-Agent Proof of Concept showcased how specialized AI agents can collaborate to automate logistics disruption management. By leveraging LangGraph, the system efficiently coordinated alert analysis, route planning, decision-making, and execution reporting.

Overall, the project illustrates the practical application of modern AI technologies in solving real-world logistics challenges through collaborative agent-based systems.

11. Future Scope

The current implementation can be further enhanced by integrating additional enterprise capabilities, including:

- Google Maps API for real-time routing.
 - Live traffic analysis.
 - Weather-based route optimization.
 - SAP ERP integration.
 - Warehouse Management System (WMS) connectivity.
 - Human approval workflow.
 - Web-based dashboard.
 - Real-time shipment tracking.
 - Autonomous AI agents with memory.
 - Cloud deployment using FastAPI and Docker.
-

12. References

1. LangGraph Documentation
 2. LangChain Documentation
 3. Groq API Documentation
 4. Google Gemini Documentation
 5. Python Official Documentation
 6. Meta Llama 3.1 Documentation
-